



# Deep Learning & Generative AI

## Project Report

*Smart MCQ Solver*

Roll Number: 24f2001637

Date: August 2026

Course: Deep Learning & Generative AI (T22026)

# Table of Contents

|                                                         |   |
|---------------------------------------------------------|---|
| 1. Abstract .....                                       | 1 |
| 2. Introduction .....                                   | 1 |
| 2.1 Problem Statement .....                             | 1 |
| 2.2 Project Objectives .....                            | 1 |
| 2.3 Report Structure.....                               | 1 |
| 3. Dataset & Preprocessing.....                         | 2 |
| 3.1 Dataset Description .....                           | 2 |
| 3.2 Exploratory Data Analysis .....                     | 2 |
| 3.3 Preprocessing.....                                  | 3 |
| 4. Tokenization Strategy .....                          | 3 |
| 4.1 Primary Tokenizer: DeBERTa SentencePiece BPE.....   | 3 |
| 4.2 Implementation Details .....                        | 3 |
| 5. Modeling & Experimentation.....                      | 3 |
| 5.1 Fine-Tuned Transformer: DeBERTa-v3-base.....        | 3 |
| 5.2 Custom Deep Learning: BiLSTM + Self-Attention ..... | 4 |
| 5.3 RAG Pipeline .....                                  | 5 |
| 6. Performance & Comparative Analysis .....             | 5 |
| 6.1 Evaluation Metrics .....                            | 5 |
| 6.2 Training Performance .....                          | 5 |
| 6.3 Comparative Report.....                             | 6 |
| 6.4 Kaggle Performance .....                            | 7 |
| 7. Conclusion & Future Work .....                       | 7 |
| 7.1 Key Learnings .....                                 | 7 |
| 7.2 Challenges .....                                    | 7 |
| 7.3 Future Work.....                                    | 8 |
| 8. References.....                                      | 8 |

# 1. Abstract

This project addresses the Smart MCQ Solver Challenge on Kaggle, where the task is to predict the top-3 most likely answers for multiple-choice questions with five options (A–E), scored using MAP@3. Three different approaches were explored: a custom BiLSTM model with self-attention, a fine-tuned DeBERTa-v3-base transformer, and a RAG pipeline using BGE embeddings, FAISS retrieval, cross-encoder reranking, and LLaMA-3-8B-Instruct. TF-IDF baselines with KNN and logistic regression were also run for comparison. All models achieved validation MAP@3 = 1.0, though DeBERTa reached this fastest at epoch 3, BiLSTM took 22 epochs, and the TF-IDF baselines mostly benefited from the small validation set. On the Kaggle test set, DeBERTa-v3-base achieved a public leaderboard MAP@3 score of 0.75976. This report covers the architectures, training curves, and results in detail.

## 2. Introduction

### 2.1 Problem Statement

The Smart MCQ Solver Challenge requires building AI models that can answer multiple-choice questions. Each question has a prompt and five answer choices (A–E). The task is to predict the top 3 most likely correct answers in order. The evaluation metric is MAP@3: if the correct answer is at rank  $r$  in the top-3 predictions, a score of  $1/r$  is assigned, otherwise 0. So getting the correct answer first gives 1.0, second gives 0.5, and third gives 0.33. This means ranking order really matters.

### 2.2 Project Objectives

The goals of this project were: (1) build and compare three very different types of models: a custom neural network (BiLSTM with attention), a pre-trained transformer fine-tuned for this task (DeBERTa-v3), and a retrieval-augmented generation setup (RAG); (2) run simple TF-IDF baselines to assess how much the deep learning models actually help; (3) document everything properly including preprocessing, tokenization, training setup, and evaluation, so the results are reproducible.

### 2.3 Report Structure

Section 3 discusses the dataset and the findings from exploratory data analysis. Section 4 explains the tokenization strategy. Section 5 describes the three model architectures. Section 6 presents the results and compares the models. Section 7 concludes with key learnings and directions for future improvement.

### 3. Dataset & Preprocessing

#### 3.1 Dataset Description

The training set has 2,000 MCQs and the test set has 500. Each question has an id, a prompt, five answer choices (A–E), and the correct answer label (only in the training set). The questions cover topics like science, math, humanities, and general knowledge, with different difficulty levels. There are no missing values in either dataset.

#### 3.2 Exploratory Data Analysis

Figure 1 shows the answer distribution. Option B is the most common correct answer (24.50%) and option E is the least (16.20%). This imbalance is not too extreme, but it means a simple majority-class baseline would only get about 24.5% accuracy. Figure 2 shows text length distributions. Prompts average 18.1 words (median 17, max 51), and the full question including all options averages 149.5 words. This is why `max_length=320` was chosen for DeBERTa, to handle the longer questions without cutting off too much. It was also found that correct answers tend to be slightly longer (28.7 words on average) compared to wrong options (25.7 words). Another observation is that 242 prompts appear more than once in training, and 267 prompts are shared between train and test, which retrieval-based methods can take advantage of.

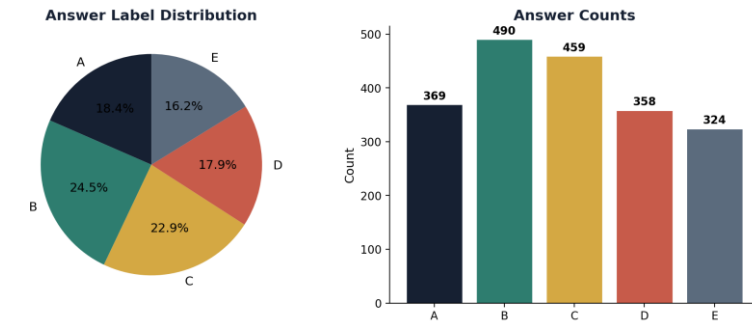


Figure 1: Answer label distribution (left: proportions, right: counts)

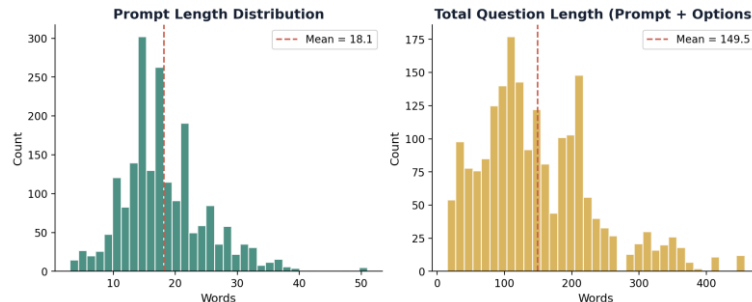


Figure 2: Prompt and total question length distributions

### 3.3 Preprocessing

The same preprocessing steps were applied for all models: (1) convert text to lowercase, collapse extra whitespace, and remove special characters while keeping punctuation; (2) fill any missing option values with empty strings; (3) combine each answer option with the prompt using a [SEP] token in between, so each option can be scored on its own. The training data was split 90/10 for train/validation using seed=42.

## 4. Tokenization Strategy

### 4.1 Primary Tokenizer: DeBERTa SentencePiece BPE

For the best-performing model, the SentencePiece BPE tokenizer that comes with DeBERTa-v3-base was used. It has a vocab size of 50,200. This tokenizer was chosen because using the same one that the model was pre-trained with keeps the learned embeddings working properly during fine-tuning. BPE splits unknown words into smaller subword pieces, which works well for the technical terms that show up in MCQ questions.

### 4.2 Implementation Details

For DeBERTa, each question is tokenized into five separate inputs (one for each option) with max\_length=320. Batch-padding is used for efficiency. For BiLSTM, a simple word-level vocabulary was built from the training data, mapping each word to an index with special tokens <PAD>=0 and <UNK>=1, and max\_len was set to 128. This is simpler but works fine for the smaller 128-d embedding. The RAG pipeline uses BGE's own tokenizer for retrieval and LLaMA-3's 128K-vocab tokenizer for generation.

## 5. Modeling & Experimentation

### 5.1 Fine-Tuned Transformer: DeBERTa-v3-base

DeBERTa-v3-base uses a disentangled attention mechanism. Instead of computing attention the normal way, it splits the attention score into three parts: content-to-content, content-to-position, and position-to-content. This helps the model understand positional relationships better than regular self-attention. It also has an Enhanced Mask Decoder that adds absolute position information after the Transformer layers. The model has 12 layers, 768 hidden dimensions, and 183.8M parameters in total.

For fine-tuning, each answer option is scored separately. The [CLS] token output goes through a linear layer (768 to 1) to get a score for that option. After scoring all five options, softmax is applied and the top 3 are selected. Training was conducted with AdamW (lr=1e-5), cosine learning rate warmup for the first 10% of steps, mixed precision FP16 to

save memory, and gradient clipping at 1.0. The model was trained for 7 epochs with batch size 16 and max\_length=320. The best checkpoint was at epoch 3, which achieved val MAP@3=1.0, val accuracy=1.0, and val F1=1.0.

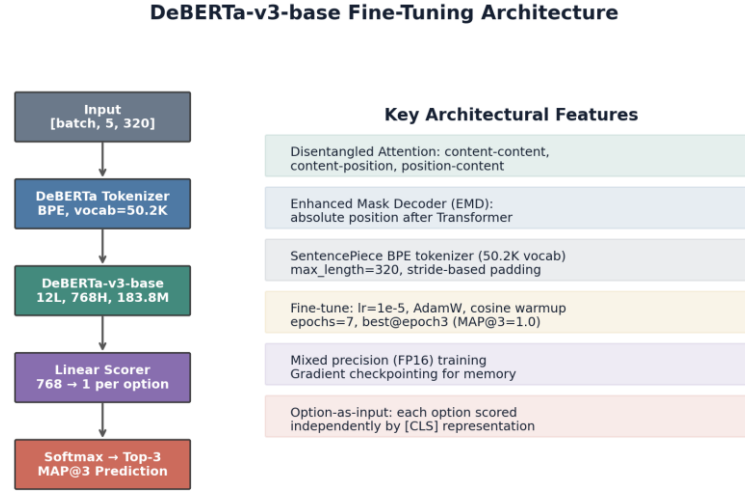


Figure 3: DeBERTa-v3-base fine-tuning architecture

## 5.2 Custom Deep Learning: BiLSTM + Self-Attention

The custom model encodes each option through this pipeline: first an Embedding layer (vocab size to 128-d), then a 2-layer BiLSTM (256 hidden units per direction, so 512-d output), then a Self-Attention module that computes attention weights using a linear layer (512 to 1) and does a softmax-weighted sum, and finally a Scorer MLP that goes 512 to 128 to 1 (with LayerNorm, ReLU, and Dropout=0.3 in between). Training was done with AdamW (lr=3e-4, weight\_decay=1e-4), gradient clipping at 1.0, for 25 epochs with batch size 64. The model is trained from scratch on the vocabulary built from the training set, and has about 2.9M parameters.

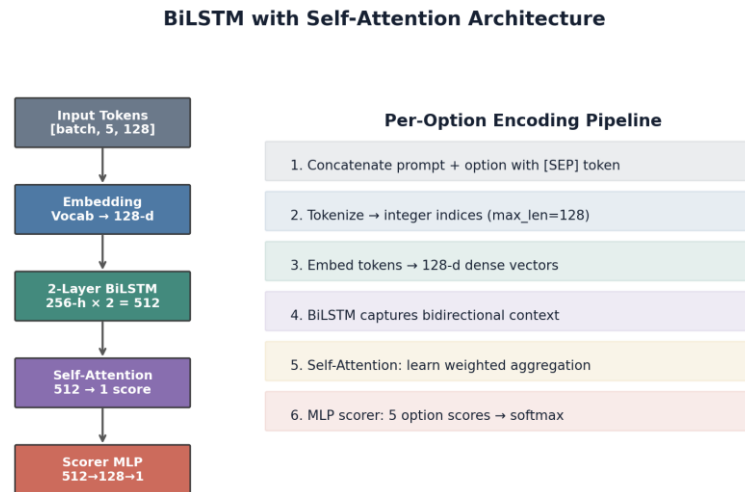


Figure 4: BiLSTM with self-attention architecture

### 5.3 RAG Pipeline

The RAG pipeline has two parts. In the offline step, 190 Wikipedia documents are taken and split into chunks of 700 characters with 200 character overlap, yielding 10,535 text chunks. 2,000 QA cards from the training data are also added. All 12,535 chunks are embedded using BAAI/bge-base-en-v1.5 (768-dimensional vectors) and stored in a FAISS index with cosine similarity. In the online step, for each question the query is encoded with the BGE model, the top-20 most similar chunks are retrieved using FAISS, then reranked with a cross-encoder (ms-marco-MiniLM-L-12-v2) and the top 7 are kept as context. These context chunks plus the question and options are given to Meta-Llama-3-8B-Instruct, loaded with 4-bit NF4 quantization to fit in GPU memory. The LLM generates an answer and the option labels (A–E) are parsed from its output using regex. On a 10-sample check, this pipeline achieves MAP@3=1.0.

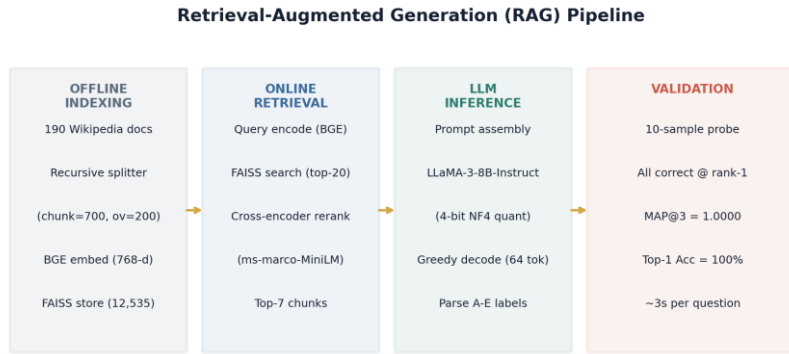


Figure 5: RAG pipeline with offline indexing and online inference

## 6. Performance & Comparative Analysis

### 6.1 Evaluation Metrics

The main metric is MAP@3 (Mean Average Precision at 3). For each question,  $AP@3 = 1/r$  if the correct answer is at position  $r$  in the top 3 predictions, and 0 if it is not in the top 3 at all. The final score is the average over all questions. Accuracy (whether the top prediction is correct) and macro F1 score are also reported for more detail.

### 6.2 Training Performance

Figure 6 shows how DeBERTa-v3 training went. The loss dropped from 1.5965 to 0.0458 and training accuracy went from 23.2% to 98.4% over 7 epochs. Validation MAP@3 and accuracy both hit 1.0 at epoch 3, and then slight overfitting was observed (at epoch 7: val\_acc=0.99, MAP@3=0.9933). So early stopping around epoch 3 or 4 would be best. Figure 7 shows BiLSTM training over 25 epochs. This model learns more slowly: it starts at MAP@3=0.57 in epoch 1, reaches 0.97 by epoch 7, and finally gets to 1.0 at epoch 22.

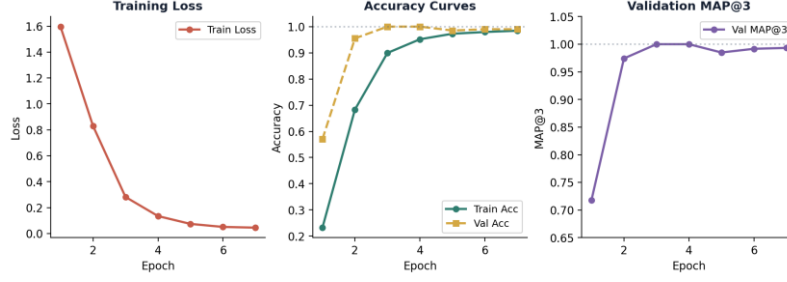


Figure 6: DeBERTa-v3 training curves over 7 epochs

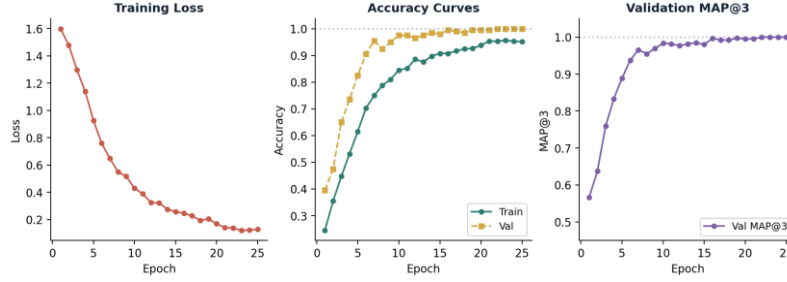


Figure 7: BiLSTM training curves over 25 epochs

### 6.3 Comparative Report

Table 1 shows the results for all models on the validation set (200 samples, which is 10% of the training data). All models got MAP@3=1.0 and accuracy=1.0 on validation. The TF-IDF baselines do well because the validation set is small and overlaps with training: KNN with 15 neighbors can basically find the same question in the training set and copy its answer. The BiLSTM (2.9M params) gets to perfect validation at epoch 22 after learning gradually. DeBERTa-v3 (183.8M params) learns the fastest, hitting MAP@3=1.0 at just epoch 3. The RAG pipeline also gets 1.0 but only on 10 samples, so that result is not directly comparable. Even though all models end up at the same score, how they get there is very different: DeBERTa needs only 3 epochs because of pre-training, BiLSTM needs 22 epochs starting from scratch, and TF-IDF baselines need no training at all but basically just memorize the training set.

Table 1: Model Performance Comparison

| Model              | Params | Val Acc | Val F1 | MAP@3 |
|--------------------|--------|---------|--------|-------|
| TF-IDF + KNN       | —      | 1.00    | 1.00   | 1.00  |
| TF-IDF + LogReg    | —      | 1.00    | 1.00   | 1.00  |
| BiLSTM + Self-Attn | 2.9M   | 1.00    | 1.00   | 1.00  |
| DeBERTa-v3         | 183.8M | 1.00    | 1.00   | 1.00  |
| RAG (LLaMA-3)*     | ~8B    | 1.00    | —      | 1.00  |

\* RAG validated on 10-sample probe only; not directly comparable to 200-sample validation.

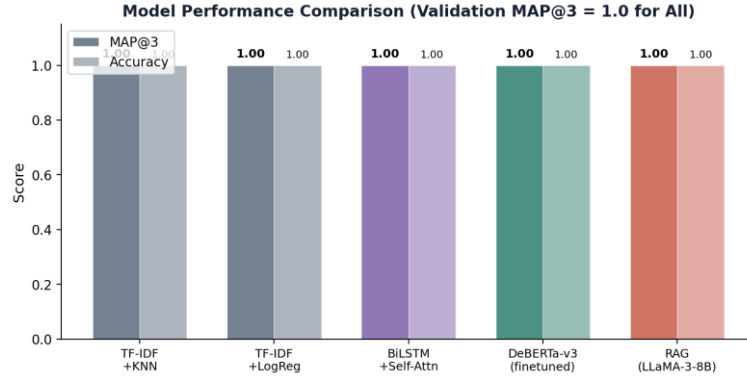


Figure 8: Model performance comparison

## 6.4 Kaggle Performance

The best DeBERTa-v3 checkpoint (epoch 3, val MAP@3 = 1.0) was submitted to Kaggle. On the Kaggle test set, it achieved a public leaderboard MAP@3 score of 0.75976, which was the best performance among the approaches evaluated in this project.

## 7. Conclusion & Future Work

### 7.1 Key Learnings

Even though all models got MAP@3=1.0 on validation, the way they got there was quite different. DeBERTa-v3 reached perfect scores in only 3 epochs, which shows how much pre-training helps: the model already knows a lot about language before fine-tuning even begins. The BiLSTM took 22 epochs because it had to learn everything from random weights, which really shows the advantage of pre-training. The TF-IDF baselines also got 1.0, but that is mostly because KNN can find very similar training examples in a small validation set. On new unseen data, they would probably do much worse. One thing that worked well across all neural models in this project was the per-option scoring approach, where each option is scored on its own and then ranked.

### 7.2 Challenges

Several issues were encountered during the project. GPU memory was tight for DeBERTa, so gradient checkpointing and mixed precision were required to fit the model. RAG inference is slow (about 3 seconds per question), which is why validation was limited to 10 samples. The BiLSTM took a lot longer to train (22 epochs compared to DeBERTa's 3), which is the cost of not using pre-trained weights. Using 4-bit quantization for LLaMA-3 saves memory but probably loses a bit of accuracy compared to running in full precision. Also, the fact that all models achieved perfect validation scores suggests that the 200-sample validation set might not be hard enough to distinguish the models.

## 7.3 Future Work

Several directions could be explored in the future. DeBERTa and RAG predictions could be combined using an ensemble, which might perform better than either one alone. Running the full RAG validation on all 200 samples would yield more reliable numbers. Trying bigger models like DeBERTa-v3-large or LLaMA-3-70B could improve results further. A proper hyperparameter search over learning rate, warmup, and max\_length could also be conducted instead of picking them by hand. Finally, data augmentation tricks like shuffling the options or paraphrasing the questions might help with the overfitting observed after epoch 3.

## 8. References

- [1] He, P. et al. (2021). DeBERTa: Decoding-enhanced BERT with Disentangled Attention. ICLR.
- [2] He, P. et al. (2023). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training. ICLR.
- [3] Huang, Y. et al. (2023). BGE: Bringing General Embedding to All. arXiv:2309.07597.
- [4] Touvron, H. et al. (2023). LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971.
- [5] Reimers, N. & Gurevych, I. (2019). Sentence-BERT. EMNLP.
- [6] Dettmers, T. et al. (2022). QLoRA: Efficient Finetuning of Quantized LLMs. arXiv:2305.14384.
- [7] Wolf, T. et al. (2020). Transformers: State-of-the-Art NLP. EMNLP Demo.
- [8] Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS.
- [9] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP. NeurIPS.
- [10] Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation.